

LMPool: 面向多GPU LLM 服务的局部性路由与NVLink KV Cache 传输

Jialiang Li
香港科技大学

Abstract

前缀缓存可以避免重复的prefill 计算，但数据并行LLM 副本的KV cache 相互独立，可复用状态可能位于并非当前选定worker 的GPU 上，而严格将请求路由到前缀owner 又可能使该GPU 过载。本文提出LMPool，一个多GPU LLM serving 系统：它以KV-aware routing 解决cache locality，以直接NVLink transfer 在KV 放置不再匹配需求时提供cache fluidity。路由器在连续已就绪前缀复用、排队成本和容量成本之间进行权衡，pair- 和payload-aware 成本模型仅在预测可节省的prefill 时间足以覆盖transfer 成本时，才准许copy 或move 操作，版本化事务协议确保未完成的副本不会参与路由决策。在六张RTX 3090 GPU 上，针对Qwen3-0.6B 和Qwen3-1.7B，5 倍共享前缀长度的routing 相比round-robin data parallelism，分别将吞吐提升11.8% 和21.3%，将平均TTFT 降低18.6% 和25.2%，并将未缓存prefill token 数降低59.9% 和61.0%。对所有layer 的打包K/V microbenchmark 验证了按GPU pair 区分的1-64 block transfer profile。端到端实验验证了background placement 以及foreground offloading 的功能，同时表明一次已完成的transfer 本身并不必然带来更低的E2E latency 或更高的throughput。源代码位于<https://github.com/0x0addc001/LMPool>。

1 引言

PagedAttention [1] 和RadixAttention [2] 使单个推理引擎中的前缀复用成为现实。然而，多GPU 部署通常在每张GPU 上运行一个数据并行模型副本和一个cache allocator。物理KV block 仍局部于各自副本，而请求则经由集群级入口到达。这种分离产生了放置问题：输入prompt 可能匹配已驻留在某张GPU 上的前缀，但传统round-robin dispatcher 可能将请求分配到另一张GPU，选中的副本因而需要重新计算并保存同一前缀。

全局cache policy 有两种相反的失败方式。将每个匹配请求都路由到prefix owner 会把热点前缀的请求集中到一个副本，使其他副本利用不足并降低整体batching efficiency。不加区分地重新分发KV block 则会产生通信、同步和内存干扰成本，即使本地重新计算更便宜也是如此。因此，有效的策略必须让request router 与KV-cache placement mechanism 共享对locality、load、capacity 和topology 的一致视图。

LMPool 遵循两条原则。Routing principle 解决cache locality：当减少的prefill work 大于额外的排队和容量压力时，请求应在已经保存相关KV prefix 的GPU 上执行，有效的routing 也会减少后续数据移动。Transfer principle 解决cache fluidity：当KV placement 不再匹配请求负载时，直接NVLink transfer 可将有用block 移到可复用它的副本。Transfer 不解决任意request skew，它解决KV 放置不均和跨实例复用，否则这些情形只能依赖数据移动或重新计算。

因此，系统在能够保持locality 时优先routing，只有数据移动既必要又有利时才使用最快的受支持直接transfer path。LMPool 将多GPU HBM 视为逻辑协调的pool，而不是物理共享内存。每个worker 保留对本地allocation 与KV tensor 的所有权，control plane 只协调metadata、reservation、routing decision 和transfer transaction，不访问物理KV 数据。

本文实现了locality- 和load-aware router，以及cost-gated KV-transfer path。入口处的router 在所有healthy replica 中匹配连续完整prefix block，并权衡节省的prefill work 与queueing、decode、capacity 和reclamation cost。Transfer path 仅限直接NVLink pair，批量处理完整block chain，支持copy 与move，并在显式transaction 中使用generation check

和rollback。实验通过三类受控workload 区分稳定的routing 收益、已验证的transfer 机制，以及transfer 无法摊销其成本的情况。

2 背景与动机

2.1 Paged KV Caching

自回归decoding 会在每个layer 为每个已处理token 保存key 和value vector。PagedAttention 将这些状态划分为固定大小的物理block，并将每个sequence 的逻辑block 映射到物理block [1]。Prefix caching 对完整逻辑block 做hash，使具有相同hash chain 的两个请求可以引用同一ready physical block。Block-based allocation 减少fragmentation，hash-based sharing 避免重复prefill，但复用仍局限于拥有该物理block 的cache allocator。对被拆分为完整block $b_0, \dots, b_k$ 的prompt，LMPool 计算chained hash:

$$h_i = \text{xxhash64}(h_{i-1} \parallel b_i), \quad (1)$$

其中 h_{-1} 为空。部分tail block 不能全局复用。有效匹配必须从 h_0 开始且保持连续，孤立的后续hash 不足以构成匹配，因为其attention dependency 不存在。

2.2 观察一：复用是放置属性

只有请求在保存匹配KV state 的GPU 上执行，或者transfer 该state 比重新计算更便宜时，prefix-cache hit 才有价值。Round-robin 下的独立副本可能保留许多本地cache entry，却在请求准入时无法使用它们。相反，global router 可在不增加物理cache capacity 的情况下提升request-level 与token-level reuse。因此，应先考虑routing，再考虑数据移动。

2.3 观察二：局部性与负载均衡竞争

长共享前缀往往对应大量请求。将每个匹配请求都送到第一个owner 会将locality 转化为load hotspot。仅看aggregate hit rate 无法暴露此问题，因此评估还必须包括每个rank 的请求数、token 数、prefill/decode time、GPU utilization 和memory utilization。Routing decision 需考虑剩余prefill work、queued token、active decode work、free block 和reclamation pressure，目标不是最大化hit rate，而是最小化expected completion cost。

2.4 观察三：Transfer 必须覆盖其成本

在评估节点上，直接NVLink 比host-mediated storage path 更快，但每次transfer 都会reserve destination block、同步两个worker、消耗link 与memory bandwidth，并更新control meta-data。短生命周期或冷KV state 应重新计算或evict，而非transfer。当完整热点chain 可能被复用、session 移到另一副本，或保留可复用状态能缓解实际local block shortage 时，transfer 才有吸引力。这一观察促成了transfer batching、基于实测bandwidth 的cost estimation，以及foreground 与background transfer 的独立admission policy。

2.5 范围

LMPool 面向同一模型在一台多GPU 节点上的副本，直接NVLink pair 可被显式配置或自动发现。由于ingress request 尚未分配给worker，ingress routing 可选择任意healthy replica。Worker-originated rerouting 与所有KV-transfer plan 则仅限当前GPU 及其已配置direct NVLink partner，不相关的PCIe 或NUMA peer 不作为transfer path。系统补充而非替代RDMA-based 和storage-tier system，cross-node RDMA、CPU/SSD tier、heterogeneous model 与透明remote memory addressing 不在当前范围内。

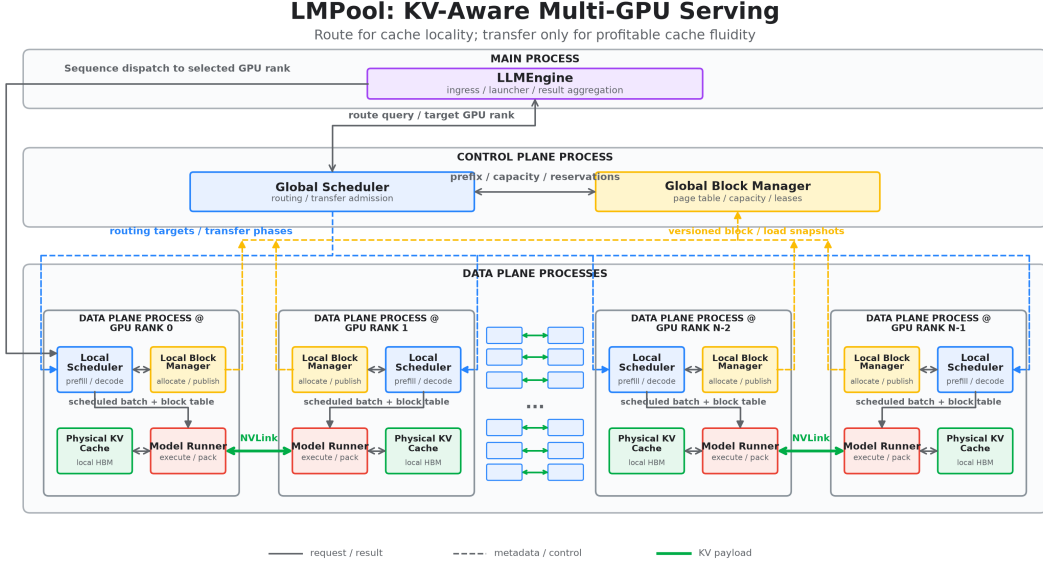


Figure 1: LMPool 的request、control、metadata 与KV data path 分别位于main process、control plane 和per-GPU data plane。Global Scheduler 向每个Local Scheduler 发送routing 和transfer command，每个Local Block Manager 向Global Block Manager 发布版本化block 与load snapshot。只有配对的Model Runner 通过NVLink path 交换打包KV tensor，物理KV data 不经过control plane。

3 系统设计

图 2-4 区分三个常被混淆的操作：routing 选择请求执行的GPU，background placement 预测需求并准入candidate replica，transactional transfer 验证并执行已准入plan。图中的decision node 展示rejection、deferral、spill 和rollback path，hotness threshold 不能绕过capacity、cooldown、pair-idleness 或measured-cost check。

3.1 进程与所有权模型

LLMEngine 是面向用户的进程。它启动一个独立control-plane process 和 N 个相同的数据-plane process，其中包括rank 0 的数据-plane process。新sequence 通过LLMEngine 进入，后者经由ControlPlaneClient 将sequence metadata 与prefix hash 发送给control plane。Control plane 在LLMEngine 向选定worker 入队完整sequence 前选择rank，worker 通常不会重复全局routing。

Control process 拥有GlobalScheduler 与GlobalBlockManager。每个data-plane process 拥有一个Local Scheduler、Local Block Manager、Model Runner 和CUDA device。这一分离定义了ownership boundary：global page table 是global metadata 的authoritative source，而每个Local Block Manager 对physical allocation、reference count、readiness 和local KV tensor 具有authoritative ownership。Global Scheduler 向每个Local Scheduler 发送routing target 与transfer phase，每个Local Block Manager 在相关state change 后向Global Block Manager 发送版本化snapshot。

Request/result path 在LLMEngine 和selected worker 之间传递完整Sequence object、generated token 与runtime measurement。Metadata/control path 向control plane 传递版本化block/load snapshot，向worker 传递transfer phase 与heartbeat command。KV data path 绕过LLMEngine 和control plane：source Model Runner 从local HBM cache gather contiguous tensor，pair-local NCCL communicator 通过direct NVLink transfer，destination Model Runner 再将其scatter 到destination Local Block Manager 保留的block。

LLMEngine 还会监控并可重启control process。双向heartbeat 检测无响应worker 或control

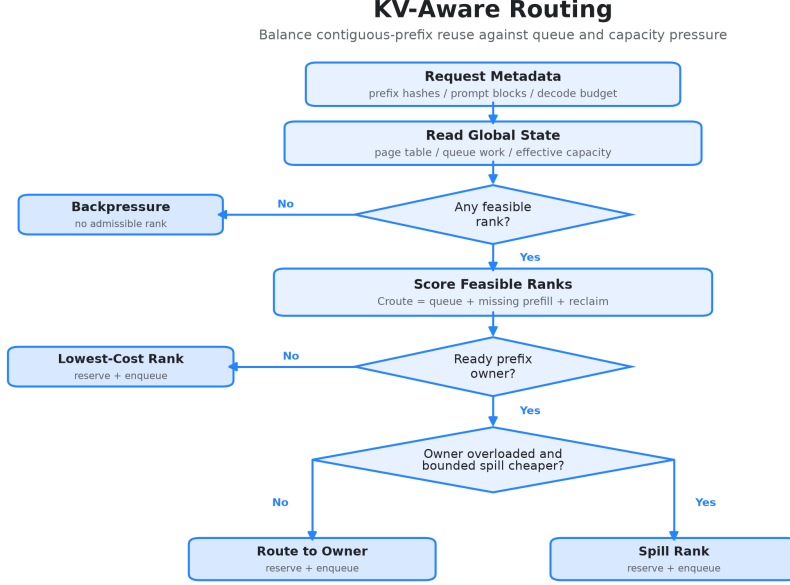


Figure 2: KV-aware routing 过滤不可行GPU，并根据连续prefix reuse、projected load 和capacity 选择rank。仅当prefix owner 的load 足够高且额外recomputation cost 受限时，bounded spill 才绕过owner。

process，但不提供consensus 或high availability。control-process restart 后可由worker state snapshot 重建control metadata，当前系统不能在launcher failure 后继续服务。

3.2 权威全局元数据

GlobalBlockManager 维护page table $h \mapsto \{(g, b, \text{generation}, \text{ready})\}$ ，可记录同一hash 的多个replica。对每张GPU，它还保存free、used、evictable block、parent relationship、recency 与access frequency 的版本化snapshot。附加state 包括routing reservation、active transfer 中的block、placement lease、worker epoch 和用于拒绝stale update 的snapshot version。

Worker 在allocation、append、release、preemption、transfer publication 或相关cache-access change 后发布complete snapshot。GlobalBlockManager 以单GPU 为粒度原子替换snapshot。Routing 时产生的reservation 会在request commit 首次prefill operation 前暂时减少可用容量，防止一批routing decision 重复消费同一stale free-block count。Heartbeat reception 不推进page-table freshness，缺失或stale snapshot 会触发对complete state 的显式请求。

3.3 KV-Aware Routing

LLMEngine 在将ingress request 分配给worker 前先提交给GlobalScheduler。将该情形记为 $s = -1$ ，此时每个healthy replica 都是candidate，topology affinity $w(-1, g) = 1$ 。对由worker 发起、rank 为 $s \geq 0$ 的routing request，candidate set 包含 s 及直接连接的NVLink partner。令 $\mathcal{N}_{\text{NV}}(s)$ 表示这些partner，则

$$w(s, g) = \begin{cases} 2, & g = s, \\ 1, & g \in \mathcal{N}_{\text{NV}}(s), \\ 0, & \text{其他情况} \end{cases} \quad (2)$$

Affinity 为零的candidate 在cost evaluation 前被移除。该区分允许global admission-time routing，同时让worker-originated rerouting 和physical data movement 局限在已配置direct pair。

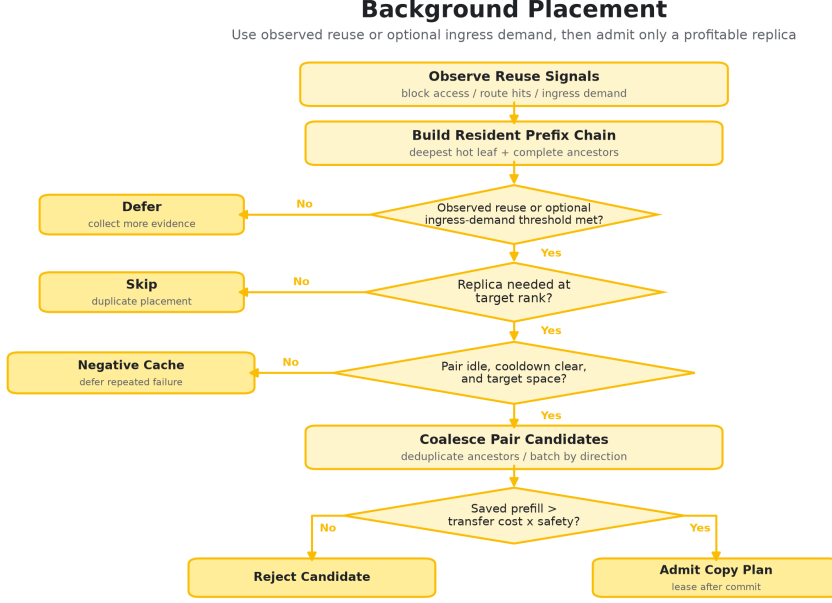


Figure 3: 仅当complete-block observation 或ingress forecast 表示存在需求，且cooldown、pair-idleness、target capacity 与measured-benefit check 均通过时，background placement 才准入candidate replica。

对candidate rank g ，令 N 为prompt token 数， R 为request block 数， h_g 为最长contiguous ready-prefix match， S 为block size， F_g 为当前free block 数。所需新block、按block 对齐的uncached prefill work 与reclamation pressure 为

$$n_g = \max(0, R - h_g), \quad M_g = \min(N, n_g S), \quad A_g = \max(0, n_g - F_g). \quad (3)$$

只有free capacity 与dependency-safe reclaimable capacity 之和覆盖 n_g 时，rank 才可行。Token-equivalent queued work 为

$$Q_g = w_w(T_g^{\text{wait}} + T_g^{\text{pending}}) + w_r T_g^{\text{running}} + w_s(N_g^{\text{running}} + N_g^{\text{pending}}), \quad (4)$$

默认 $w_w = 1$ 、 $w_r = 0.25$ 、 $w_s = 32$ 。GlobalScheduler 最小化

$$C_{\text{route}}(g) = Q_g + w_p M_g + w_e A_g S, \quad (5)$$

默认 $w_p = 1$ 、 $w_e = 0.5$ 。因此locality 经由 h_g 直接降低缺失prefill work，而非作为从cost 中扣除的独立hit-rate reward，prefix 与topology score 仅用于同成本candidate 的tie break。

Router 选择cost 最低的可行candidate。当owner 的sequence pressure 比另一个rank 高出配置阈值且额外recomputation cost 有界时，它可以绕过prefix owner。当没有candidate 含有可复用prefix 时，router 选择projected cost 最低且effective capacity 足够的rank。该策略缓解request skew 和replica underutilization，而不改变底层数据并行执行策略。

3.4 本地分配与回收

Local Block Manager 首先搜索匹配的ready local block，命中后增加reference count。若未命中，它从free deque 分配block identifier，并只在Model Runner 写入KV data 后发布complete block。完成的prefix 在reference count 降至零后仍可cache。Reclamation 仅考虑dependency-safe leaf block，避免移除ancestor 导致保留descendant 孤立。Replacement policy 结合access

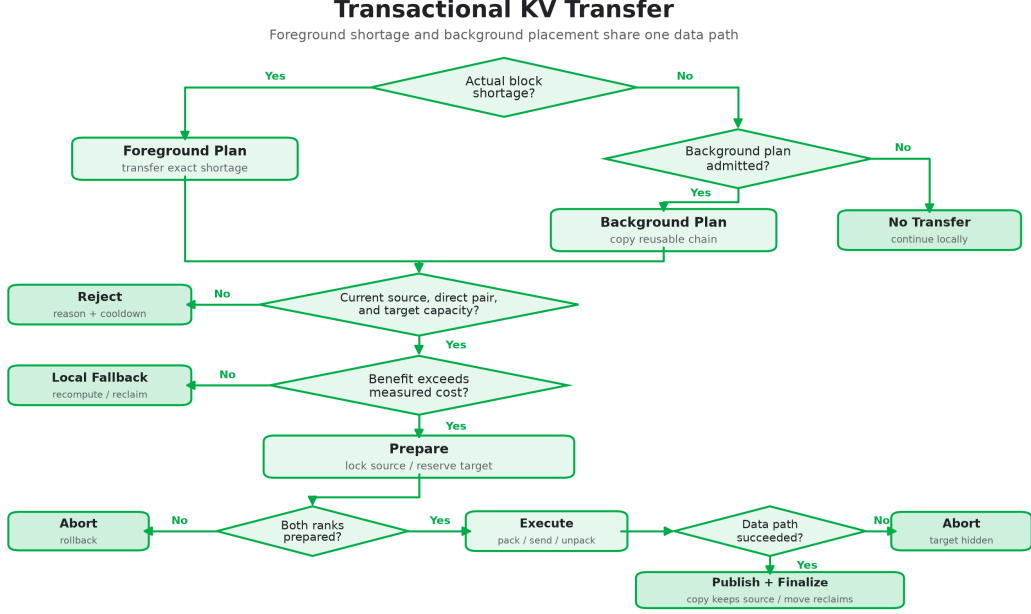


Figure 4: Transactional transfer 对foreground block shortage 或已准入background chain 依次执行source validation、benefit check、preparation、pair-local NCCL transfer、publication 和finalization。任一失败阶段均走abort path，不暴露destination。

frequency 与 recency。reference count 为正的live block 不能作为move-based eviction 的对象，但在replication 有利时可被copy。图 6 概括normal local path 和transactional cross-GPU path。

“received” 与 “published” 的区别定义了visibility boundary。成功的NCCL receive 本身不创建可路由replica，destination 必须先local snapshot 中记录complete hash chain 与readiness state，之后control plane 才可暴露新位置。Copy semantics 下，被引用的source 保持有效，只会被unlock，move semantics 下，仅unreferenced、dependency-safe suffix 可被reclaim。partial 或尚未ready 的local block 在release 后绕过cache，直接回到free deque。

3.5 Foreground Transfer

Foreground transfer 由需求驱动。若local prefill 或decode append 期间出现 k 个block 的shortage，worker 向control plane 请求恰好 k 个block，而非该请求的完整block count。GlobalScheduler 选择complete source chain 并决定操作为move 或copy。Publication 后，move 可在source 释放dependency-safe suffix，copy 则保留source 并创建另一个replica。Pinned prefix 可以copy，但不能释放。

对 B 个block、 L 个layer、block size S 、 H_{kv} 个KV head、head dimension D 与每元素 d byte，打包payload 为 $X = 2BLSH_{kv}Dd$ 。对每个logical NVLink pair p ，transfer microbenchmark 提供单调P95 data-path profile $\mathcal{P}_p = \{(X_i, Q_{p,i}^{95})\}$ ，另一profile $\mathcal{R}_p = \{(X_i, R_{p,i}^{95})\}$ 记录已完成serving transaction 的P95 非负dispatch-to-publication residual。Data-path prior 与static estimate 为

$$T_{\text{data}}^p(X) = \text{PWL}(X, \mathcal{P}_p), \quad (6)$$

$$T_{\text{static}}^p(X) = (R_p^{95}(X) + IT_{\text{data}}^p(X))w_x. \quad (7)$$

模型在相邻sample 间做线性插值，低于测量范围使用最近boundary，高于范围则延伸最后一段slope。令 $b(B) = 2^{\lceil \log_2 B \rceil}$ 为size bucket，已完成source operation 和dispatch-to-publication

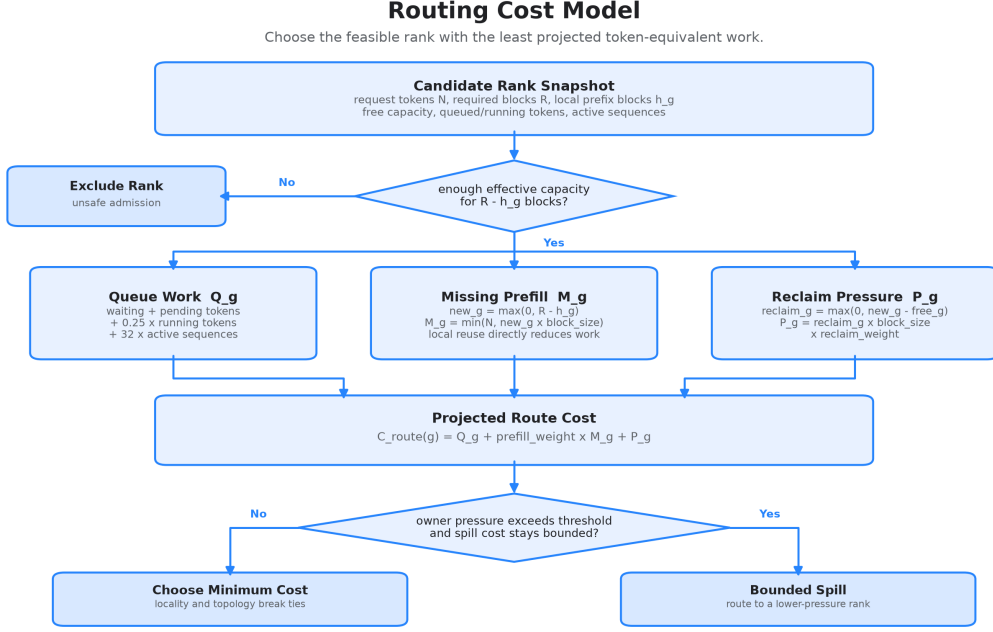


Figure 5: Routing cost 对每个capacity-feasible rank 合并token-aware queued work、block-rounded uncached prefill 与dependency-safe reclamation pressure。仅当prefix owner 的load 足够高、额外recomputation cost 在配置限制内时允许bounded spill。

observation 分别更新非负residual EWMA $\Delta_{p,b}^{src}$ 与 $\Delta_{p,b}^{place}$ 。Conservative admission estimate 为

$$T_{\text{xfer}} = \max\{T_{\text{prior}}^p, (T_{\text{data}}^p + \Delta_{p,b}^{src})w_x, (IT_{\text{data}}^p + \Delta_{p,b}^{place})w_x\}. \quad (8)$$

其中 $I = 1.2$ 为interference multiplier, w_x 为配置的transfer-cost weight。有calibrated residual profile 时 $T_{\text{prior}}^p = T_{\text{static}}^p$, 无此profile 时, 先使用zero-residual fallback, 直至出现completed placement observation, 之后在data-path estimate 与online placement estimate 中取较大者。Residual profile 与EWMA 均为非负, 异常快observation 不会使admission 比calibrated data-path prior 更激进。Foreground saved work 使用discounted historical chain reuse $\hat{R}$, background admission 只计入第一次可避免的cold prefill:

$$T_{\text{save}}^{fg} = \hat{R}BSt_{\text{prefill}}, \quad T_{\text{save}}^{bg} = BSt_{\text{prefill}}(g_{dst}). \quad (9)$$

只有source generation 与target capacity 仍有效且 $T_{\text{save}} \geq \rho T_{\text{xfer}}$ 时plan 才被准入。具有相同low-benefit 或insufficient-capacity signature 的重复background candidate 会进入negative cache, 直到signature 变化。

3.6 Background Placement

Background placement 解决可预测的未来reuse, 而不是即时allocation failure。Worker 为ready block 报告cumulative access count 与parent hash。Control plane 按最深hot leaf 选择maximal chain, 并按prefix order 重建所有resident ancestor。Route-hit count 提供online trigger, 若ingress 知道尚未提交的queued request, 也可提供从hash 到future demand 的forecast。Forecast 可用时预测reuse 受其上界限制, 否则, 预测reuse 由chain 上minimum access count 减去observed use 并采用conservative discount 得到。该信号只产生candidate, admission 还要求destination 不存在replica、directed pair 的cooldown 和idle-pressure check 完成, 且target capacity 足够。

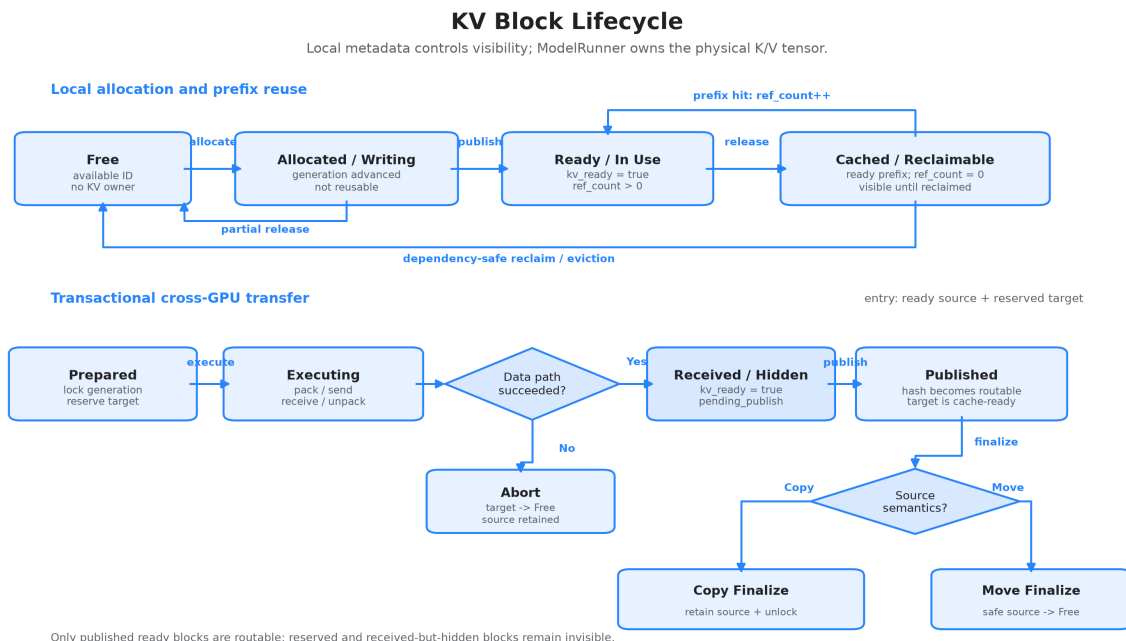


Figure 6: KV block 在Local Block Manager 所有权下 经历allocation、readiness、reference、caching、reservation、publication 和reclamation state。Model Runner 写入或transfer 物理K/V tensor, reserved 或received 但未published 的block 对routing 不可见。Publication 先于copy 或move finalization, abort 保留source 并释放destination reservation。

Compatible candidate 按directed NVLink pair 去重并batching。默认每个candidate chain 至多八个block, 每个coalesced pair payload 至多128 个block, 这只是上界, 不是固定transfer size。Background admission 最多计入destination 第一次可避免的cold prefill, 因为没有中间eviction 时destination 会在第一次miss 后变warm。Placement lease 只在publication 后创建, source 与replica 的明确quota 将后续matching request 分配到两张GPU, 将speculative placement 与routing 耦合, 而非仅复制数据。

Load-skew workload 在无rank hint 情况下测试background placement。Warm-up phase 为每个hot prefix 创建一个instance, 后续burst 重用这些prompt identity。Control plane 从prefix ownership、direct topology、capacity 和load state 选择source/target rank, placement lease 随后将完成的replica 暴露给routing。

3.7 Transactional Transfer Protocol

Transfer plan 有四个外部可见phase。Preparation 中, destination reserve 具体free block ID, 两个worker 记录plan state, routing 不能观察新位置。Execution 中, source/destination 发起匹配的NCCL point-to-point operation。Source 用indexed selection 将physical block gather 成形状为 $[L, 2, B, S, H_{kv}, D]$ 的contiguous tensor, 分别表示layer、K/V、transferred block、block token、KV head 与head dimension, 预初始化的pair-specific process group 一次transfer 该tensor。Destination receive 同形状buffer, 并用indexed copy 将数据scatter 到其Local Block Manager reserve 的物理block。Hash、generation 和destination block ID 是control protocol 中的metadata, 不进入NCCL tensor payload。

Publication 中, destination 将received block 标为ready 并发布versioned snapshot, 只有此后control plane 才可暴露新的page-table location 或placement lease。Finalization 中, move 会在释放指定suffix 前验证每个source block 的hash 与generation, copy 则保留source。Preparation 或execution 失败会abort transaction、释放所有reservation, 并保留source authority。

Protocol 对每个plan ID 和phase 都是idempotent。Per-client receive lock 串行化response

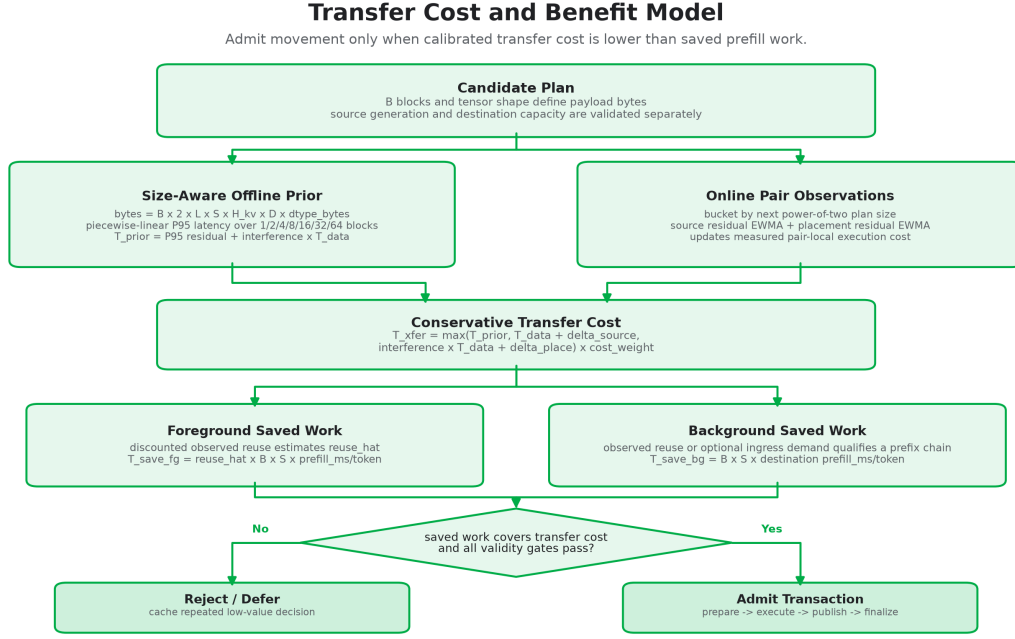


Figure 7: Transfer admission 将expected saved prefill time 与相应size bucket 的pair-specific P95 latency profile 及runtime residual 对比。Foreground 与background path 对saved work 的estimate 不同，但共享validity、capacity 和benefit-ratio check。

Table 1: 主要实现模块

模块	职责
LLM engine	Request ingress、process launch/supervision、completion aggregation
Control plane	Protocol handling、client、authoritative event loop、heartbeat processing
Global scheduler	基于cost 的routing 与transfer-plan decision
Global block manager	Global page table、snapshot、reservation、candidate tracking
Data plane	Per-rank worker loop 与transfer-phase execution
Local scheduler	Local prefill/decode admission 与shortage detection
Local block manager	Physical block、prefix cache、reference 与reclamation
Model runner	Model execution、KV tensor、metric 与transfer interface
KV transfer	打包NCCL GPU-to-GPU payload movement

queue 消费，一个event loop 串行化control-plane state mutation，每个worker 内block mutation 保持single-threaded。这些措施保护metadata transaction，同时不在inference fast path 上引入global lock。

4 实现

LMPool 使用Python 与PyTorch 扩展单GPU 推理引擎Mini-vLLM。表 1 总结了implementation。

4.1 测试策略

我为每个engine module 实施了unit test。表 2 总结coverage。测试使用fake component 获得deterministic control decision。

Table 2: 测试设计与覆盖范围

测试组	覆盖行为
Block manager	Allocation/release、chain reuse、LFU/LRU leaf、generation、snapshot
Global scheduler	Hit/miss/capacity route、load bypass、direct-pair filter、move/copy plan
Control plane	Epoch、stale update、reservation、lease、idempotent phase、heartbeat、concurrency
Data plane/engine	Queue protocol、master-first ingress、worker/control lifecycle、result aggregation
KV transfer	Shape/byte accounting、copy/move primitive、two-rank NCCL equality、deadlock detection
Model runner	KV layout、prefill/decode preparation、dtype consistency、transfer adapter
Benchmark	Workload、schema、metric、confidence interval、plotting、profile construction/validation
End-to-end	Multi-process completion 与 optional CUDA/NCCL integration

5 评估

评估回答四个问题: (Q1) KV-aware routing 是否在长共享前缀下改善service? (Q2) Transfer data path 提供何种latency/bandwidth? (Q3) Background placement 是否在load skew 下执行, 它带来什么serving trade-off? (Q4) Foreground offloading 是否能在memory skew 下安全且高效地工作? Q1 是主要端到端性能主张, Q2-Q4 证明data-path correctness、protocol behavior 与completed transfer 不产生增量服务收益的边界。

5.1 设置与方法

评估在一台七GPU host 中选取六张24 GiB GeForce RTX 3090。实测topology 有三组direct pair: physical GPU (0,1)、(3,4) 和(5,6), benchmark 将其分别暴露为logical pair (0,1)、(2,3)、(4,5)。NVLink topology 与peer-access matrix 均显示每个selected pair 内为direct path、pair 间无direct path。NV4 表示四条bonded link, 而非fourth-generation NVLink。RTX 3090 使用GA102 的third-generation NVLink interface, 每条link 单向14.0625 GB/s, 因此一组four-link pair 理论单向峰值为56.25 GB/s, 双向合计112.5 GB/s [3]。KV-transfer workload 是单向的, 因此与56.25 GB/s 上限比较。

使用BF16 的Qwen3-0.6B 与Qwen3-1.7B, 两者均有28 layer、八个KV head 和128 head dimension, 即每token 114,688 KV byte。KV block size 为256 token。每个system experiment 使用每worker 固定KV block budget, 以seed 0 重复五次, 禁用EOS termination。Routing、memory skew、load skew 分别生成64、16、8 个output token, goodput 使用workload-specific E2E SLA。每个JSON artifact 记录command、resolved model configuration、Git state、individual trial value、95% confidence interval 与per-rank statistic。每个scenario 在fresh process 中运行, cache content 不跨scenario 保留, 完整artifact 对应experiment data ‘20260727T231622Z’。

5.2 合成数据集剖析

研究使用deterministic synthetic prompt, 而非public conversational trace。每个prompt 将Qwen chat template 应用于repeated instruction body 和group-specific suffix, 使可复用KV state 的数量与ownership 显式可控。表 3 对tokenized input 与phase structure 进行profiling, 仅有request hit rate 不足以量化可复用prompt token。

在应用任何system policy 前, 报告两个intrinsic trace metric。令 m_i 为请求 i 中cumulative hash 已出现在先前请求、且连续完整的prefix block 数, B 为block size, T_i 为prompt length。Request prefix sharing 为 $\sum_i \mathbb{1}[m_i > 0]/N$, token prefix sharing 为 $B \sum_i m_i / \sum_i T_i$ 。这

些数值是unlimited cache capacity 与perfect placement 下的ordered-replay upper bound, 且不含partial tail block, runtime request hit 与cached-token ratio 另行报告。

Table 3: 合成工作负载剖析

工作负载	请求数	前缀构造	Prompt token	请求共享	Token 共享	Window	目的
Routing	192	16 个recurring group, 每组首请求为cold, 共享前缀长度为1x、3x、5x	365,880 / 1,084,728 / 1,803,576	91.67%	86.20% / 91.38% / 89.93%	16	测试locality-aware routing。
Load skew	192	三个long group, 三个warm-up 后为189 次hot reuse	1,444,344	98.44%	97.15%	96	测试background copy 与placement lease。
Memory skew	72	六个session warm-up、30 个anchor/cold pressure request、36 个session reuse	486,554	91.67%	72.29%	48	测试move-style source-capacity release。

5.3 Baseline 与指标

定义五个normalized scenario: single-gpu 运行一个replica, multi-gpu 使用static round-robin data parallelism, multi-gpu-kv-routing 开启global routing、关闭transfer, multi-gpu-kv-transfer 保持round-robin request placement、开启transfer, multi-gpu-lmpool 同时开启两种机制。所有multi-GPU scenario 每worker 使用相同KV block budget。这些scenario 是受控内部消融, 隔离routing/transfer 的增量作用, 并保持model execution、KV capacity 与request generation 不变, 它们不是与production vLLM、SGLang、Mooncake 或LMCache deployment 的直接比较。

主要指标为output throughput、SLA goodput、mean TTFT、decode time per output token (TPOT)、mean/P90 E2E latency 和uncached prefill token。对请求 i , 令 s_i 为submission time、 f_i 为first-token time、 c_i 为completion time、 N_i 为output token 数, 则 $TTFT_i = f_i - s_i$, $TPOT_i = (c_i - f_i)/(N_i - 1)$ 。 $N_i = 1$ 的请求无decode interval, 不计入TPOT statistic。该定义排除queueing/prefill time。另报告control-plane request hit、route-to-owner hit、data-plane local request hit 与cached-token ratio。Transfer counter 报告block/payload byte、copy/move operation、protocol outcome、estimated/observed cost, per-rank request、token、execution time 和utilization 分布揭示load concentration。

对 R 次完整重复测得的metric, 报告sample mean $\bar{x} = R^{-1} \sum_{r=1}^R x_r$ 与双侧95% confidence interval $\bar{x} \pm t_{0.975, R-1} s / \sqrt{R}$, 其中 s 为sample standard deviation。实验 $R = 5$, $t_{0.975, 4} = 2.776$ 。对P90 E2E 等tail metric, x_r 是第 r 次重复中的request-level P90, 区间则在五个repetition-level P90 间计算。这些区间量化固定设置下run-to-run variation, 而非request-level latency distribution 或model/hardware 间变化。

5.4 Q1: KV-aware routing 是否在长共享前缀下改善service?

Routing 收益随前缀长度增大。图 8 报告五次重复均值与95% confidence interval。1x 时, routing 不改善throughput: Qwen3-0.6B 从164.7 变为161.8 tok/s, Qwen3-1.7B 从148.9 变为150.9 tok/s, TTFT 也不变或略高。匹配前缀太短, 避免的prefill 尚不足以主导正常scheduling variation。3x 与5x 时, 避免的work 足以改变service metric。5x 下, 0.6B throughput 从43.63 提升到48.77 tok/s, mean TTFT 从536 降至436 ms, mean E2E 从2,743 降至2,338 ms, P90 E2E 从3,977 降至3,140 ms, uncached prefill token 从884,997 降至354,821。1.7B throughput 从38.22 提升到46.38 tok/s, mean TTFT 从691 降至517 ms, mean E2E 从3,156 降至2,479 ms, P90 E2E 从4,903 降至3,649 ms, 对应uncached prefill 降低61.0%。长前缀设置下的改善超过报告的run-to-run confidence interval。

Locality 被转换为节省的计算。Routing sweep 中request-level sharing bound 恒为91.67%, 但token sharing 随共享前缀变长而增加。Router 不为hit rate 添加独立奖励, 更长的contiguous

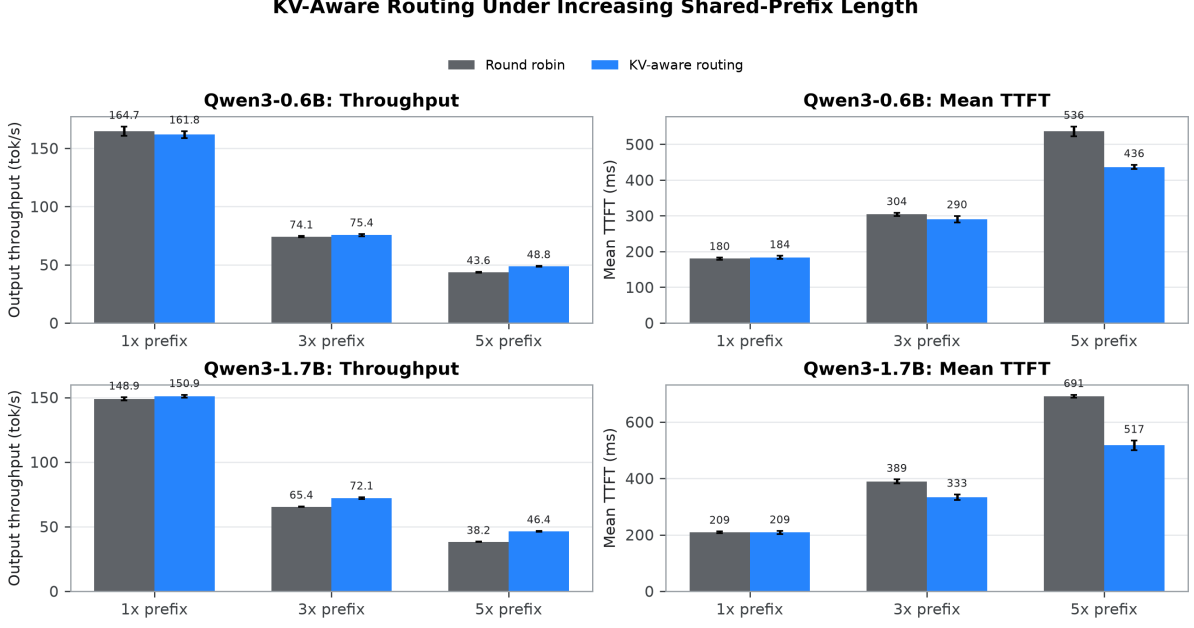


Figure 8: 随共享前缀长度增加的Routing 结果

match 降低route cost 中的missing-prefill term, 因此同一locality 直接减少scheduled compute。5x 时效果最强, 两个模型同时改善throughput 与所有报告latency metric, 而非只改善cache-hit counter。

5.5 Q2: Transfer data path 提供何种latency/bandwidth?

Data path 保持KV 语义。 Microbenchmark 在两rank 间transfer 每个layer 的K/V tensor, 并逐byte 验证destination。它验证packed tensor shape、byte count、dtype path 和receive-side content, 说明bandwidth measurement 对应serving path 实际使用的tensor, 而非无关communication primitive。Microbenchmark 本身不证明端到端serving benefit, 因为它不包含future reuse、queueing 或model execution interference。

大plan 摊销固定data-path cost。 图 10 给出serving path 使用的同一packed tensor 的effective bandwidth。Profile 覆盖三个direct pair、两个模型和1/2/4/8/16/32/64-block plan, 逐byte 验证destination, 并初始化pair-specific P95 piecewise-linear latency curve。64-block payload 约为1.75 GiB, mean critical-path latency 约48 ms, 其bandwidth 高于small plan, 是因为gather、packing、communicator synchronization 与scatter 被更多K/V data 摊销, 不应将该曲线视为raw-link measurement。Runtime plan 不限于采样size, scheduler 按payload 插值并以已完成operation 更新pair-by-size residual。

5.6 Q3: Background placement 是否在load skew 下执行, 它带来什么serving trade-off?

Load-skew trace 覆盖预期路径。 请求共享率98.44% 的load-skew trace 在三次warm-up 后留下target capacity。两个模型中, transfer-only 和完整LMPool 均完成三次background placement, copy 87 个block, 完整LMPool 随后通过placement lease 路由95 个请求, 而transfer-only 保持round-robin admission。这表明copied prefix 对后续routing 可见, 而非仅完成data-path operation。

TTFT 改善, 但decode 可限制E2E。 对0.6B, LMPool 将mean TTFT 从round robin 的507 ms 降至237 ms, 将uncached prefill token 从152,568 降至41,208。相对routing-

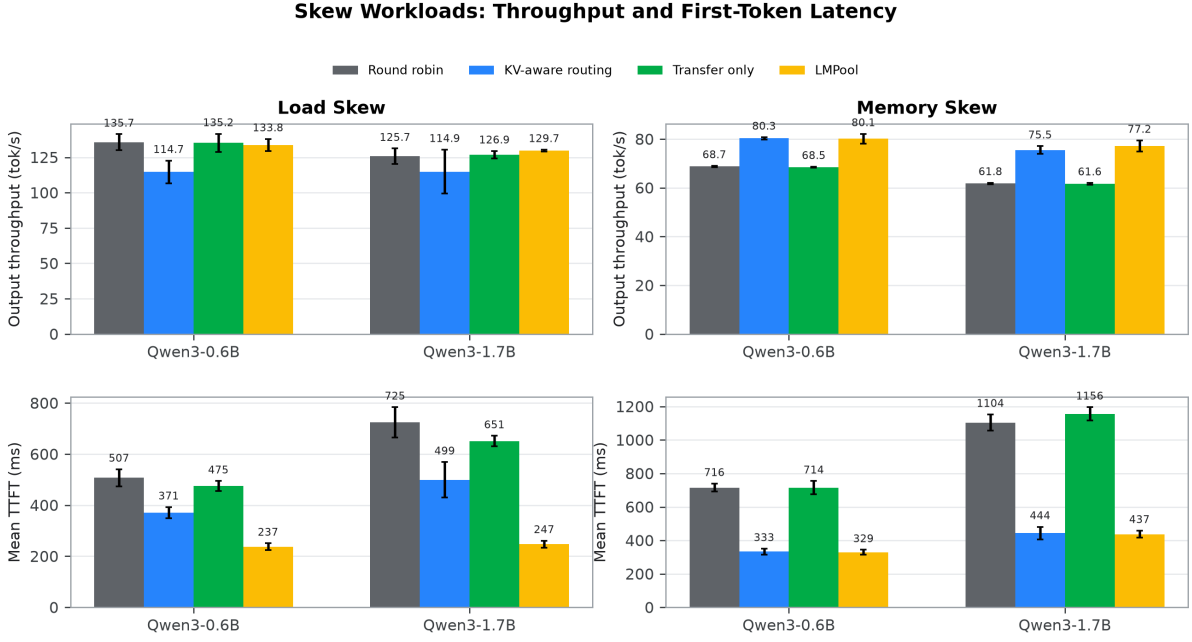


Figure 9: Load-skew 与Memory-skew 工作负载上的Throughput 和Mean TTFT

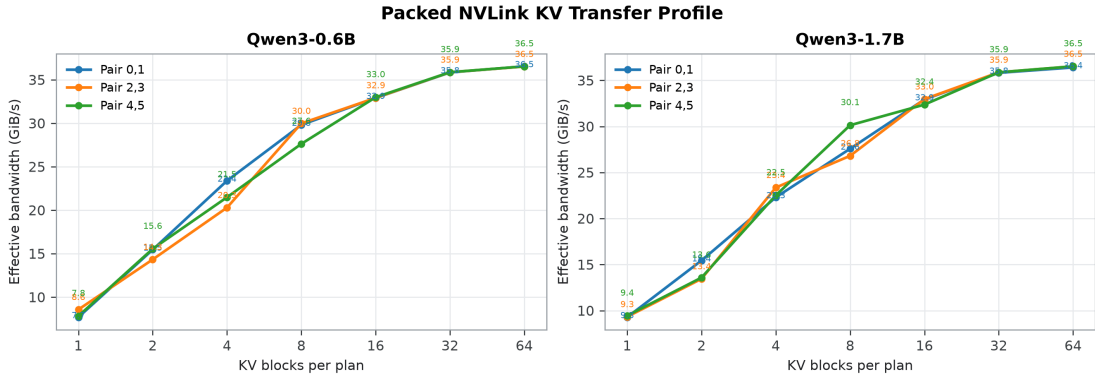


Figure 10: 跨Direct GPU Pair 的打包NVLink KV Transfer Profile

only, TTFT 从371 降至237 ms, throughput 从114.7 增至133.8 tok/s, 这与lease routing 使用copied replica、而非让请求停在original owner 的解释一致。但它仍略低于round-robin throughput (133.8 对135.7 tok/s), 且mean E2E 更高 (2,754 对2,540 ms)。对1.7B, LMPool 将throughput 从125.7 增至129.7 tok/s, TTFT 从725 降至247 ms, P90 E2E 从4,850 降至4,071 ms, mean E2E 从2,782 略增至2,840 ms。该E2E trade-off 与mean TPOT 增加一致: copied-prefix placement 降低prefill delay, 但当前decode scheduler 可能在first token 后引入更多contention。因此, load-skew 结果支持background-copy correctness 和显著TTFT benefit, 而非universal mean-E2E claim。

5.7 Q4: Foreground offloading 是否能在memory skew 下安全且高效地工作?

Offloading 被验证功能, 但未必可以仅凭自身提高性能 Memory skew 中, 0.6B full-system 结果报告35 个释放的source block、两次copy operation 和true offload verification flag。Transfer-only 与round robin 接近, throughput 为68.47 对68.75 tok/s, TTFT 为714 对716 ms, 说明movement 本身没有移除足够work 以改善service。Routing 是主要改善来源: 它通过选择已有local prefix chain, 将throughput 从68.75 提升至80.30 tok/s, TTFT 从716 降至333 ms, 加入move path 后为80.11 tok/s 和329 ms, 与routing-only 在统计上接近。1.7B 结果记录了moved

block, 但不满足完整offload verification predicate, 其full-system throughput/latency 与routing-only 接近, confidence interval 重叠。该实验验证安全的capacity-release transaction, 但未证明增量throughput 收益。因此, memory skew 被用于implementation/safety result, 而非证明transfer-out 改善serving performance。Memory pressure 本身不能保证存在可复用、可执行且足够有价值的transfer plan。

6 讨论、局限与未来工作

本研究仅使用一台host、六张consumer GPU 和两个小于2B 参数且KV geometry 相同的模型, 未评估超过六张GPU 的scale-out 或对256-token KV block size 的sensitivity。较小block 可改变fragmentation、metadata cost、transfer-plan size 与batching efficiency, 较大模型改变compute/transfer ratio, 较大NVLink fabric 改变candidate selection。NVSwitch fabric 需要单独评估: 它扩大每个source GPU 的可行target set, 且并发transfer 会争用共享fabric, 而非独立的pair-local link。因此, 在当前拓扑上测得的pair-specific profile 不能证明all-to-all NVSwitch domain 上的routing 或admission behavior。结果不应直接外推到RDMA 或PCIe-only cluster。

内部消融提供受控mechanism comparison, 但不比较完整production serving stack 在production traces 上。Kernel、scheduler、prefix index、connector implementation 和workload 的差异使这些改善不能被解释为相对vLLM、SGLang、Mooncake 或LMCache 的superiority。Versioned snapshot、transactional transfer、heartbeat 和control-process restart 保持metadata consistency 并支持failure detection, 但不实现replicated consensus, launcher failure 会停止service, worker failure 可能失去该worker 上的physical cache。

Model/pair-specific size curve 和bucketed EWMA 捕获payload scale 与observed serving overhead。当前runner 从pair-specific P95 data-path profile 与measured transaction residual 初始化static estimate, 只在出现placement observation 前使用zero-residual fallback。评估未报告held-out prediction error, 也未报告safety ratio/EWMA update sensitivity。Cost model 也不显式条件化concurrent kernel、GPU clock rate 或link contention, compute state 的快速变化可能导致estimate 不准确。

未来应在相同model、arrival process 与KV budget 下用production traces 比较production serving engine, 并改变model size、KV geometry、block size 与GPU count。Transfer study 应报告held-out prediction error, 并对piecewise-linear profile、EWMA correction、safety ratio 与no-gate policy 做ablation。还应在eight-GPU NVSwitch domain 上评估, 包括concurrent transfer profile、较大target set 上的topology-aware candidate selection, 以及shared-fabric contention 下的end-to-end scaling。Production deployment 还需要replicated control metadata 与launcher failover, 而非当前single-control-process recovery model。

7 相关工作

7.1 LLM Serving 与Prefix Caching

Orca 引入transformer serving 的iteration-level scheduling [4]。vLLM 的PagedAttention 将KV allocation virtualize 为block [1], SGLang 的RadixAttention 为structured program 组织reusable prefix state [2]。Sarathi-Serve 采用chunked prefill 控制prefill/decode interference [5], DistServe 分离两个phase, 并在TTFT/TPOT constraint 下优化goodput [6]。这些系统建立local execution/scheduling 的背景。LMPool 则研究data-parallel replica 间的admission-time placement, 它们可通过direct NVLink copy KV block。

7.2 Cluster 编排

NVIDIA Dynamo 是datacenter-scale inference framework, 结合disaggregated serving、基于worker load 与KV-cache overlap 的KV-aware routing, 以及通过KV Block Manager 实现

的tiered KV offload [7]。其router 和cache event 支持cluster-level worker selection, storage hierarchy 将KV capacity 扩展到GPU HBM 之外。LMPool 解决更窄的scale-up 问题: 在由configured direct NVLink pair 连接的data-parallel replica 间协调物理局部HBM cache。LMPool 的重点是根据pair 与payload 的cost 对complete KV block chain 的transactional copy/move 准入, 并以placement lease 服务后续request。二者是互补而非可相互替代的设计, 本文也不比较其end-to-end performance。

Ray [8] 与Kubernetes [9] 在cluster 或replica 粒度管理resource allocation、worker placement、scaling 和lifecycle。Ray 调度distributed task 与actor, Kubernetes 调度并维护container。它们可作为LMPool 的外层deployment layer, 但不会原生维护prefix-level KV ownership、依据contiguous ready-prefix chain 路由request, 或准入transactional KV movement。因此, LMPool 与cluster orchestration 互补: orchestrator 管理replica lifecycle, LMPool 管理一个serving group 内的request 与KV placement。

7.3 Distributed 与Tiered KV Cache

Mooncake 提出KV-cache-centric disaggregated architecture, 以distributed storage capacity 换取更少的prefill computation [10]。vLLM 与Mooncake Store 的较新集成通过GPUDirect RDMA 和distributed DRAM/SSD service 在instance 间pool KV state。其agentic trace 官方评估在12 张GB200 上报告3.8× 更高throughput、46× 更低P50 TTFT 和8.6× 更低E2E latency, synthetic scale-out 在60 GPU 上保持超过95% cache hit rate [11]。该集成可在scheduling 时查询block hash, 但作者将完整cache-aware routing 与multi-node NVLink support 列为future work。PegaFlow 则把cache ownership 放在long-lived Rust service, 结合pinned host DRAM、one-sided RDMA read 和local SSD, vLLM/Novita 报告在八个Qwen3-8B instance 共享一个fixed host-cache budget 时throughput 提升56%, deduplicated TP8 MLA KV 时提升72%, 8-NIC RDMA cluster 中至少1 GiB object 的平均read bandwidth 为194 GB/s [12]。

MemServe 提供elastic memory pool 和global prompt-tree scheduler 用于disaggregated serving [13], CachedAttention 在hierarchy 中保存multi-turn KV state, 并与compute overlap load/save [14], CacheGen 压缩KV state 以在受限link 上加速context loading [15], LMCache 提供跨engine 与storage/communication backend 的reusable cache layer [16]。这些系统强调capacity、persistence 或cross-node visibility。LMPool 关注更窄的scale-up setting: 由direct NVLink 连接的data-parallel replica 上物理局部的HBM cache。

7.4 Migration 与Multi-GPU Memory Balance

Aqua 在bursty memory pressure 下利用由NVLink/NVSwitch 连接GPU 的spare memory, 对inference state 做page, 以支持preemptive、time-sliced serving。在八张H100 上, Aqua 报告最高20× 的responsiveness 提升和单个long prompt 最高4× throughput 提升 [17]。Aqua 因而为active inference state 提供fluidity, 但不按content-addressed prefix ownership 路由请求。Llumnix 通过迁移request 支持dynamic rescheduling 与defragmentation [18], MELL 联合考虑multi-GPU KV load 与request migration cost [19]。这些研究表明state movement 是有成本的scheduling operation, 而非总是有利的fallback。LMPool 的区别在于使用content-addressed prefix chain、copy-based replication 与placement lease, 使source 与destination 都可服务未来请求。

表 4 区分常被统称为“distributed KV cache” 的三类关切。Locality-aware placement 将请求分配到已有相关prefix 的GPU, fluidity-aware placement 在当前位置不再匹配需求时移动state, coupled admission 在routing/transfer 间选择前比较recomputation、queueing、capacity 与measured transfer cost。表中“Partial” 表示metadata lookup, 但没有公开的policy 联合考虑locality、load、routing 与transfer admission。比较并不表示LMPool 替代durable 或cross-node cache system, 而是说明在所列系统中, LMPool 将三种decision 集成在serving scheduler 内。

Table 4: 按Locality-Aware Placement、Fluidity Mechanism、Durable Capacity 和Coupled Admission 的KV-Cache 系统比较

系统	主要范围	Locality-aware request placement
Mooncake Store [11]	Cluster-wide agentic prefix reuse	否: 提供cache-location information 给scheduling 来作metadata lookup, 但不路由requests to the owner by itself 或联合考虑locality、load、routing 与transfer admission。
PegaFlow [12]	External、process-independent KV lifetime/sharing	否: vLLM 保留request scheduling, global index 只定位remote hit
Aqua [17]	GPU-memory burst 下responsive preemption	否: 无content-addressed prefix routing
LMPool	Data-parallel GPU replica 间的KV locality/fluidity	是: 最长contiguous prefix 与load/effective capacity 联合评分
系统	Fluidity mechanism/path	Durable/multi- Coupled route-transfer admission tier pool
Mooncake Store [11]	GPU HBM 与distributed cache worker 间的asynchronous GPUDirect RDMA	是: DRAM/SSD否: 无公开模型比较recomputation 与transfer cost
PegaFlow [12]	Local CUDA IPC、cross-node one-sided RDMA; cold block 使用SSD	是: pinned 否: TinyLFU 管理cache admission, DRAM、remote 不在routing/transfer 间选择 DRAM、SSD
Aqua [17]	经NVLink/NVSwitch 将inference state page 到spare GPU HBM	否: volatile GPU HBM 否: 基于memory pressure, 而非expected future prefix reuse
LMPool	在configured NVLink pair 的peer HBM 间copy/move	否: volatile GPU HBM 是: 先route, 再基于measured cost 与expected reuse 准入transfer

8 结论

LMPool 通过global routing 协调物理局部KV cache 以获得cache locality, 并仅在expected benefit 覆盖cost 时使用pair-local NVLink transfer 获得cache fluidity。实现将authoritative control process 与per-GPU data process 分离, 并在configured direct pair 上结合versioned metadata 与transactional block movement。实验在两个模型的长共享前缀下显示稳定routing gain, 验证了load skew 中的background placement 及memory skew 中foreground offloading 的source-block release 的功能, 但completed movement 本身不保证更低E2E latency 或更高throughput。

致谢

感谢Yijun Sun、Kai Chen 教授的项目指导, 以及Mini-vLLM 作者提供基础推理引擎。

References

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.
- [2] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024. doi: 10.52202/079017-2000.
- [3] NVIDIA Corporation. NVIDIA Ampere GA102 GPU Architecture. Technical report, NVIDIA Corporation, 2020. URL <https://nvidia.com/en-us/data-center/ampere-ga102/>

[//www.nvidia.com/content/dam/en-zz/Solutions/geforce/ampere/pdf/NVIDIA-ampere-GA102-GPU-Architecture-Whitepaper-V1.pdf](https://www.nvidia.com/content/dam/en-zz/Solutions/geforce/ampere/pdf/NVIDIA-ampere-GA102-GPU-Architecture-Whitepaper-V1.pdf).

- [4] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- [5] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/agrawal>.
- [6] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.
- [7] NVIDIA. NVIDIA Dynamo documentation, 2026. URL <https://docs.nvidia.com/dynamo/>. Accessed August 2, 2026; software available at <https://github.com/ai-dynamo/dynamo>.
- [8] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 561–577, 2018. URL <https://www.usenix.org/conference/osdi18/presentation/moritz>.
- [9] The Kubernetes Authors. Kubernetes documentation: Kubernetes scheduler, 2026. URL <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>. Accessed July 31, 2026.
- [10] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a KVCache-Centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, 2025. URL <https://www.usenix.org/conference/fast25/presentation/qin>.
- [11] Yifan Qiao, Trong Dao Le, Ao Shen, Zhewen Li, and Bowen Wang. Serving agentic workloads at scale with vLLM x Mooncake, May 2026. URL <https://vllm.ai/blog/2026-05-06-mooncake-store>. Software available at <https://github.com/kvcache-ai/Mooncake>.
- [12] Novita AI and the vLLM Team. vLLM x Novita AI: PegaFlow for production-grade external KV cache, May 2026. URL <https://vllm.ai/blog/2026-05-18-pegafLOW>. Software available at <https://github.com/novitalabs/pegafLOW>.
- [13] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. MemServe: Context caching for disaggregated LLM serving with elastic memory pool. *arXiv preprint arXiv:2406.17565*, 2024. URL <https://arxiv.org/abs/2406.17565>.

- [14] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024. URL <https://www.usenix.org/conference/atc24/presentation/gao-bin-cost>.
- [15] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56, 2024. doi: 10.1145/3651890.3672274.
- [16] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025. URL <https://arxiv.org/abs/2510.09665>.
- [17] Abhishek Vijaya Kumar, Gianni Antichi, and Rachee Singh. Aqua: Network-accelerated memory offloading for LLMs in scale-up GPU domains. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 48–62, 2025. doi: 10.1145/3676641.3715983. URL <https://arxiv.org/abs/2407.21255>. Software available at <https://github.com/aquam1/aqua>.
- [18] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 173–191, 2024. URL <https://www.usenix.org/conference/osdi24/presentation/sun-biao>.
- [19] Qianli Liu, Zicong Hong, Fahao Chen, Peng Li, and Song Guo. Mell: Memory-efficient large language model serving via multi-GPU KV cache management. *arXiv preprint arXiv:2501.06709*, 2025. URL <https://arxiv.org/abs/2501.06709>.